
COMP1816 - Machine Learning Coursework Report

Maria Nour Abbad – 001296459

Word Count: No more than 3500 words (3345)

1. Executive Summary

The rapid growth of food delivery platforms has increased the importance of efficient operations and reliable service. Accurately understanding delivery performance and anticipating customer dissatisfaction are key challenges for improving both operational efficiency and customer experience. This report analyses a food delivery dataset to identify the factors influencing delivery time and to develop predictive models for delivery performance and customer complaints.

The dataset contains 1,000 historical delivery orders, including features describing order characteristics, pricing, system workload, and temporal information available at the time of order creation. The analysis begins with exploratory data analysis (EDA) to examine data distributions, relationships between variables, and key patterns affecting delivery outcomes. This is followed by clustering to identify underlying operational scenarios within the data, providing insight into different system conditions such as high demand or routine operations.

Building on these insights, machine learning models are developed for two predictive tasks. The first task focuses on predicting delivery time using regression techniques, while the second task aims to classify whether a customer complaint will occur. Both tasks emphasise the use of appropriate preprocessing, feature selection, and evaluation strategies to ensure realistic and unbiased modelling.

The findings of this report demonstrate how operational factors such as system load, order size, and timing influence delivery performance. While predictive models provide useful insights, their performance also highlights the limitations of the available data, indicating that additional contextual factors may be required for more accurate predictions. Overall, the report illustrates the role of data-driven approaches in supporting decision-making within food delivery operations.

2. Data Exploration

2.1 EDA

Dataset overview:

The dataset consists of historical food delivery orders and contains approximately 1000 records with 16 features, including both numerical and categorical variables describing order characteristics, pricing, operational conditions, and temporal information. Key variables include order size (`total_items`), order value (`subtotal`), and delivery system workload (`total_outstanding_orders`, `total_busy_partners`), along

with temporal attributes such as day of the week and order time. A sample of the dataset is shown in Fig.1.

	market_id	store_primary_category	order_protocol	total_items	subtotal	\
0	AREA B	japanese	Protocol 1	4	875	
1	AREA A	dessert	Protocol 1	5	5345	
2	AREA A	burger	Protocol 5	4	2150	
3	AREA A	thai	Protocol 5	5	3570	
4	AREA D	dessert	Protocol 1	3	2485	

	min_item_price	max_item_price	total_onshift_partners	\
0	175	350	82.0	
1	295	1995	50.0	
2	50	1300	32.0	
3	175	1195	61.0	
4	295	1195	75.0	

	total_busy_partners	total_outstanding_orders	created_day	created_holiday	\
0	79.0	104.0	Saturday	NO	
1	51.0	68.0	Friday	NO	
2	37.0	56.0	Tuesday	NO	
3	61.0	73.0	Sunday	NO	
4	71.0	107.0	Monday	NO	

	created_at	actual_delivery_time	complaint	complaint_id
0	05:04:03	05:38:01	NaN	NaN
1	02:27:48	03:15:36	NaN	NaN
2	04:15:41	05:29:14	NaN	NaN
3	03:43:41	04:14:56	YES	1.0
4	04:14:36	05:14:07	NaN	NaN

Fig.1 first five rows of dataset

The dataset includes a mix of integer, floating-point, and categorical (object) data types, reflecting both quantitative measurements and descriptive attributes. Some features, particularly those related to operational conditions (e.g., delivery partners and outstanding orders), contain missing values, which must be handled during preprocessing. Additionally, the complaint-related fields are highly incomplete, as they are only recorded when a complaint occurs.

A key derived feature, delivery time, was computed as the difference between order creation and delivery completion timestamps. This variable serves as the target for the regression task and provides a direct measure of delivery performance. Furthermore, the presence of a binary complaint indicator enables the analysis of customer dissatisfaction and supports the classification task.

Dataset information is shown in Fig.2. and key statistics calculated from the dataset are show in Fig.3.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   market_id                            997 non-null    object
1   store_primary_category                974 non-null    object
2   order_protocol                        997 non-null    object
3   total_items                           1000 non-null   int64
4   subtotal                             1000 non-null   int64
5   min_item_price                        1000 non-null   int64
6   max_item_price                        1000 non-null   int64
7   total_onshift_partners                920 non-null    float64
8   total_busy_partners                  920 non-null    float64
9   total_outstanding_orders              920 non-null    float64
10  created_day                           1000 non-null   object
11  created_holiday                       1000 non-null   object
12  created_at                            1000 non-null   object
13  actual_delivery_time                  1000 non-null   object
14  complaint                             244 non-null    object
15  complaint_id                          244 non-null    float64
dtypes: float64(4), int64(4), object(8)
memory usage: 125.1+ KB
```

Fig.2 Dataset information

	total_items	subtotal	min_item_price	max_item_price	\
count	1000.000000	1000.000000	1000.000000	1000.000000	
mean	3.147000	2674.94200	688.048000	1141.950000	
std	2.385599	1859.07752	496.879802	513.886087	
min	1.000000	275.00000	0.000000	169.000000	
25%	2.000000	1445.00000	299.000000	799.000000	
50%	3.000000	2134.00000	600.000000	1095.000000	
75%	4.000000	3429.00000	976.000000	1400.000000	
max	24.000000	13019.00000	4400.000000	4400.000000	

	total_onshift_partners	total_busy_partners	total_outstanding_orders	\
count	920.000000	920.000000	920.000000	
mean	45.660870	43.020652	60.033696	
std	33.481422	32.330324	53.531173	
min	0.000000	0.000000	0.000000	
25%	18.000000	16.000000	19.000000	
50%	39.000000	36.000000	42.000000	
75%	65.000000	65.000000	89.000000	
max	158.000000	146.000000	277.000000	

	delivery_time
count	1000.000000
mean	47.273500
std	17.060248
min	16.116667
25%	35.416667
50%	44.266667
75%	55.783333
max	133.433333


```

--- Median Values ---
total_items          3.000000
subtotal            2134.000000
min_item_price       600.000000
max_item_price      1095.000000
total_onshift_partners  39.000000
total_busy_partners   36.000000
total_outstanding_orders 42.000000
delivery_time        44.266667
dtype: float64

```

Fig.3 Summary of key statistics

Key Feature Distributions:

The distributions of key numerical variables (Fig. 4-5) show that most orders are small, with `total_items` typically between 1 and 4, and moderate order values (subtotal). Both variables are right-skewed, with a small number of large orders, indicating that while most deliveries are simple, some may require significantly more time.

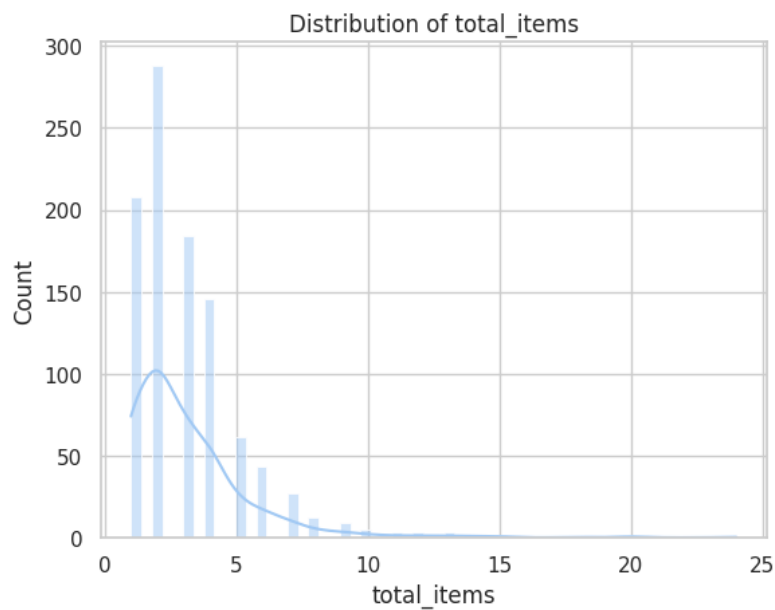


Fig.4 distribution of total_items

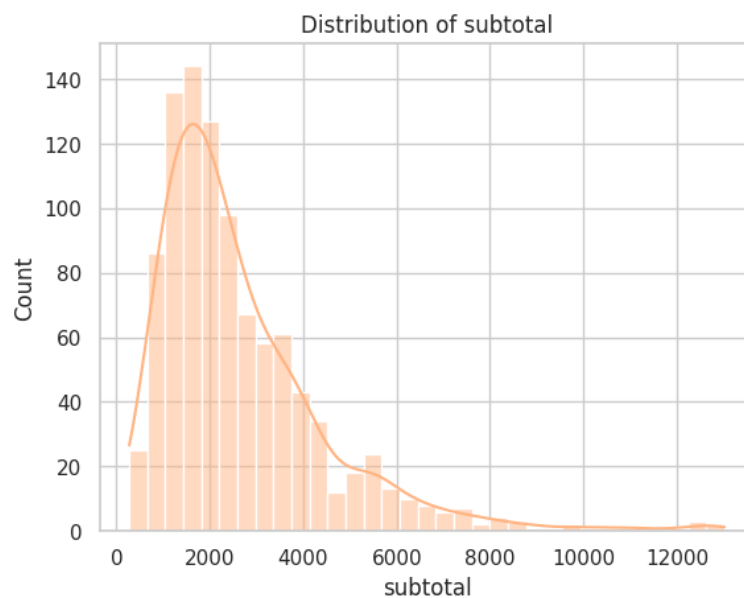


Fig.5 Distribution of subtotal

Operational variables such as `total_outstanding_orders` as shown in Fig.6 and `total_busy_partners` as shown in Fig.7 are also right-skewed, suggesting that the system generally operates under low to moderate load, with occasional periods of high congestion.



Fig.6 distribution of `total_outstanding_orders`

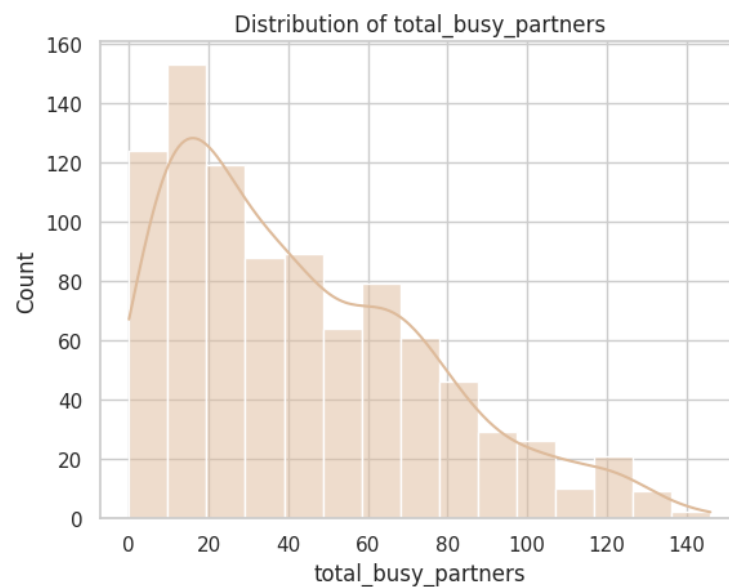


Fig.7 distribution of `total_busy_partners`

The distribution of delivery time (Fig.8) indicates that most deliveries are completed within 30–60 minutes, with a small number of longer delays.



Fig.8 distribution of delivery_times

The distribution of orders by day (Fig.9) shows higher demand during weekends, particularly on Saturday and Sunday, indicating potential increases in system load during these periods. In contrast, very few orders occur on public holidays (Fig.10), suggesting this feature has limited but potentially situational relevance.

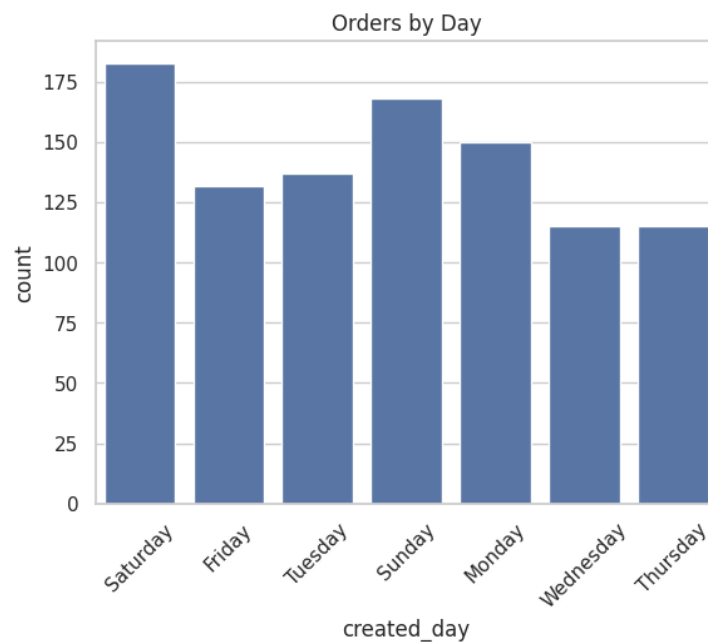


Fig.9 distribution of Orders by Day

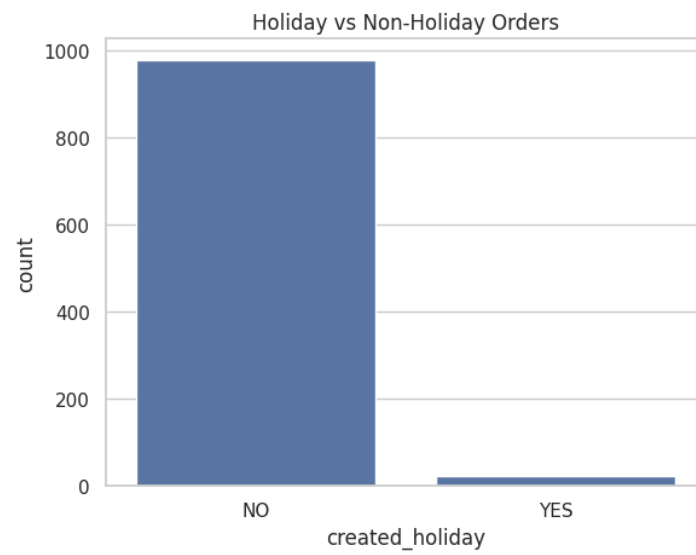


Fig.10 distribution of holiday vs non-holiday

Correlation and relationships:

The correlation matrix (Fig.11) shows strong relationships between total_onshift_partners, total_busy_partners, and total_outstanding_orders, indicating that these variables collectively represent system load. Among all features, total_outstanding_orders shows the strongest relationship with delivery time, followed by subtotal and total_items.

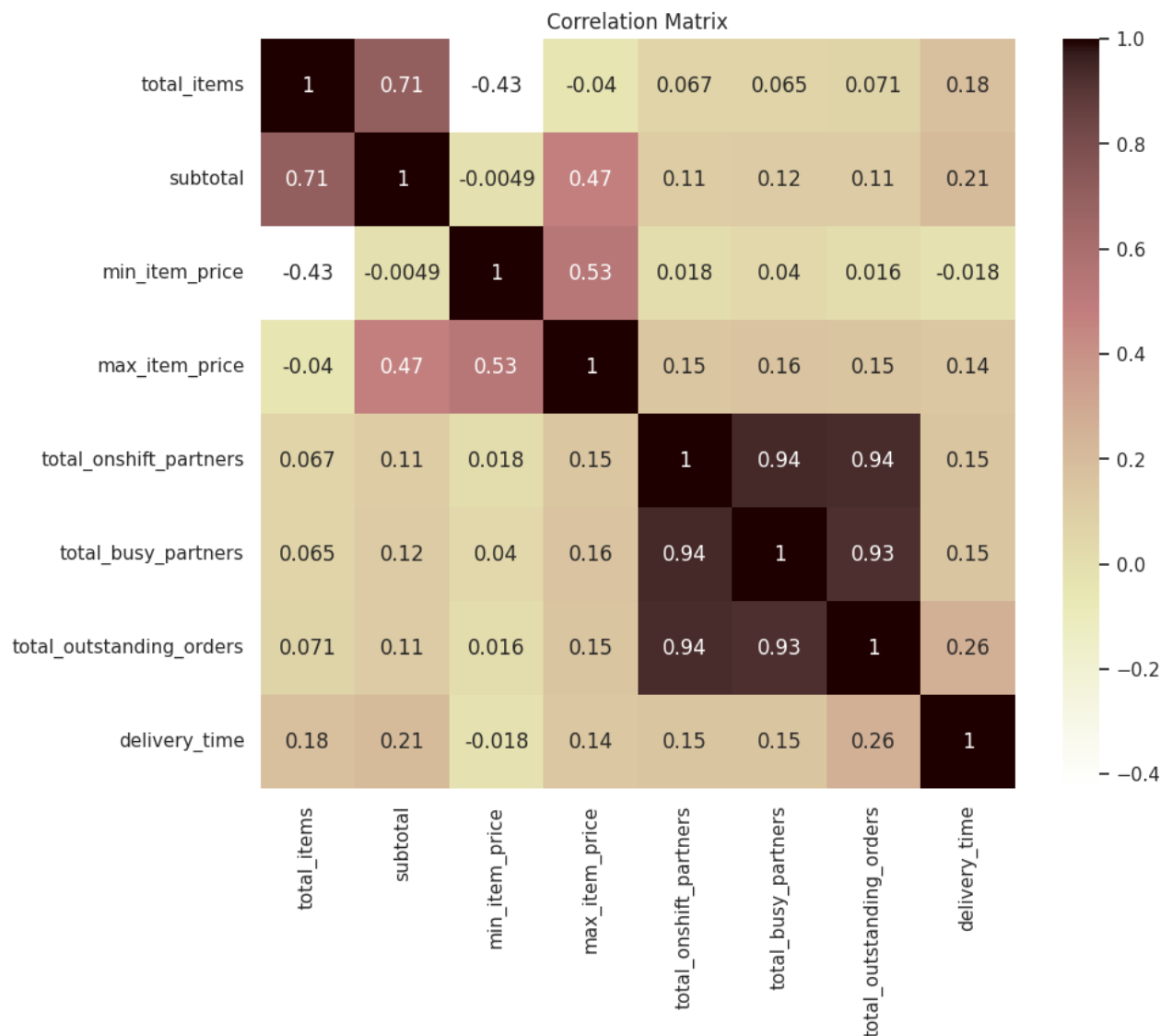


Fig.11 Correlation Matrix

Scatter plots (Fig. 12–13) further confirm that delivery time increases with system load, particularly as the number of outstanding orders rises. A weaker but noticeable trend is observed between order value (subtotal) and delivery time, suggesting that larger orders contribute to delays, although to a lesser extent than operational factors.

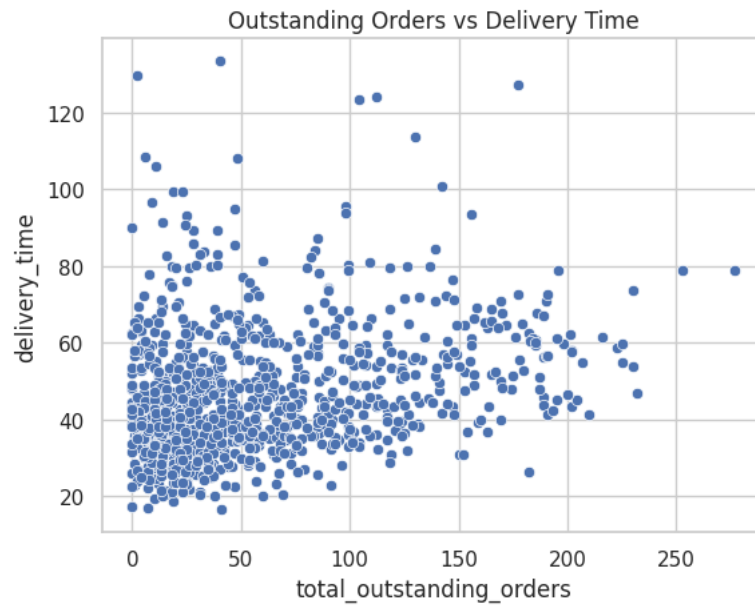


Fig. 12 Scatter plot of total_outstanding_orders vs delivery_time

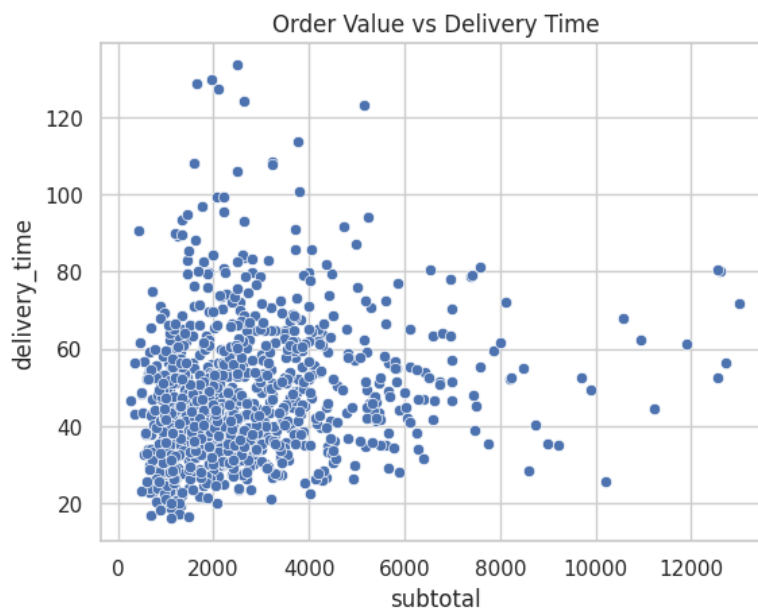


Fig. 13 Scatter plot of Subtotal vs delivery_time

The relationship between delivery time and complaints (Figure 14) indicates that longer delivery times are associated with a higher likelihood of customer complaints. Additionally, higher system load conditions also correspond to increased complaints, suggesting that delays driven by congestion negatively impact customer satisfaction.



Fig. 14 Delivery time vs complaint

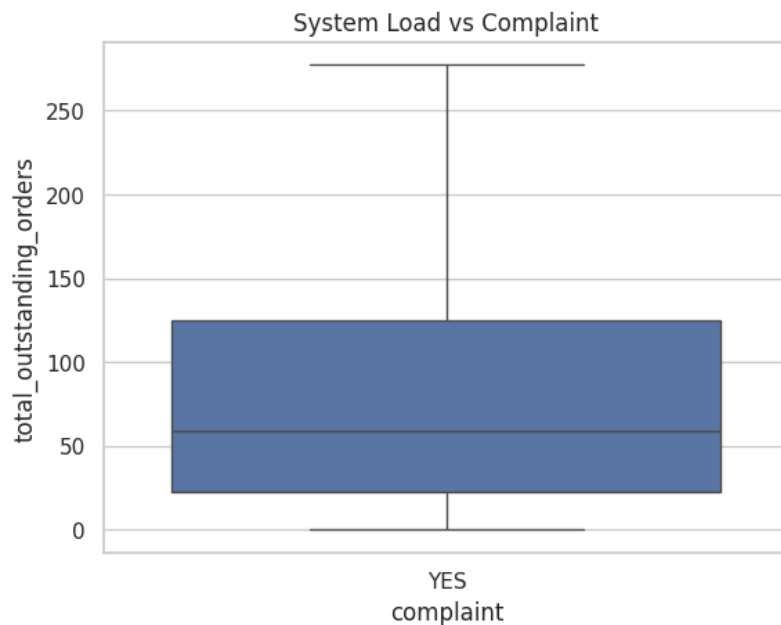


Fig. 15 complaint vs total outstanding orders

Overall, the analysis indicates that delivery time is primarily influenced by system load and order size. In particular, `total_outstanding_orders` emerges as the most important factor, highlighting the impact of congestion on delivery performance. Larger orders also contribute to increased delivery time, although their effect is less pronounced.

These findings suggest that operational variables and order size should be prioritised in predictive modelling. Furthermore, the strong association between delivery delays and complaints supports the use of delivery-related and system load features in the classification application.

2.2 Clustering

Clustering Method Selection:

To identify representative order types, K-Means clustering was applied.

K-Means is a partitioning and centroid-based clustering algorithm that assigns each observation to one of k clusters based on its distance to a cluster centroid.

The algorithm aims to minimise the within-cluster sum of squared distances:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

where C_i represents cluster i and μ_i is its centroid. The algorithm iteratively assigns data points to the nearest centroid and updates centroids using the mean of assigned points until convergence.

K-Means was selected because the dataset contains numerical features describing order size and system load, making a distance-based method appropriate. Additionally, the algorithm is efficient and produces clusters that are easy to interpret in an operational context.

Feature Selection and Preprocessing:

Clustering was performed using five numerical features:

- `total_items`, `subtotal` (order characteristics)
- `total_onshift_partners`, `total_busy_partners`, `total_outstanding_orders` (system load)

These features were selected based on the EDA, which showed that order size and operational load are key drivers of delivery behaviour. Features such as delivery time and complaint information were excluded, as clustering aims to group order types rather than use outcome variables.

Before clustering, missing values in the selected numerical features were imputed using the median.

Median imputation was chosen because several operational variables were right-skewed, making the median more robust to extreme values than the mean. The features were then standardised so that variables measured on different scales contributed equally to the distance calculations.

Determining the Number of Clusters

The optimal number of clusters was determined using both the elbow method and silhouette analysis (Fig. 16 and 17). The elbow plot shows a sharp decrease in inertia from $k = 2$ to $k = 3$, after which the rate of improvement slows significantly.

Silhouette scores further support this choice, with the highest value observed at $k = 3$. This indicates that clusters are well-separated and internally cohesive at this value.

Based on these results and the interpretability of the clusters, $k = 3$ was selected.

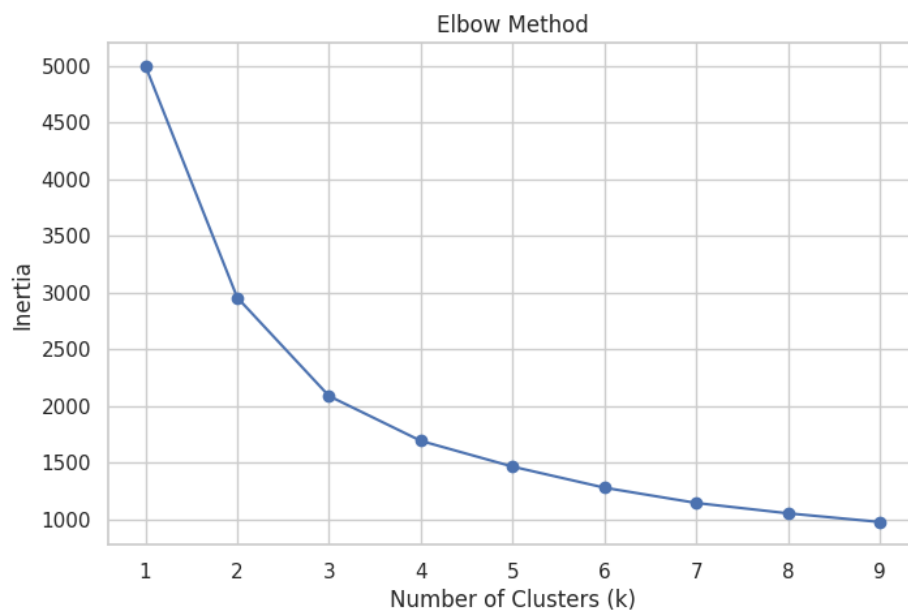


Fig.16 The elbow method

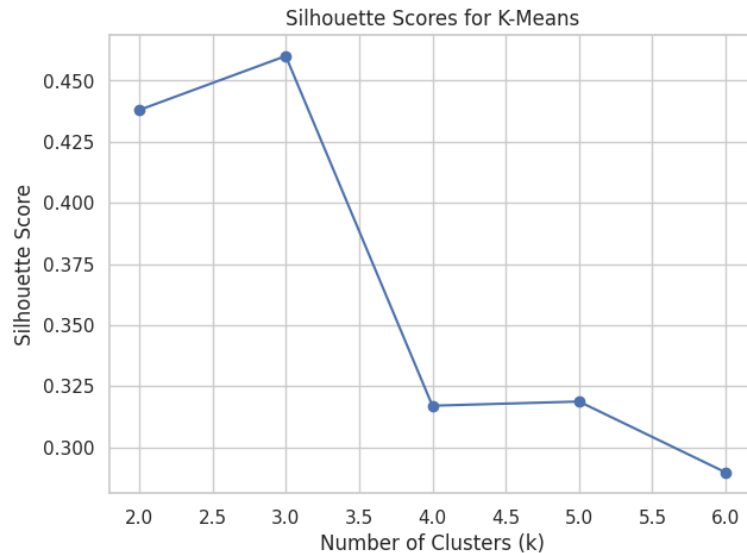


Fig.17 Silhouette scores

Interpretation:

The clustering analysis (Figures 18–19) identifies three distinct operational scenarios driven by system load and order size.

Cluster 0 – High Load Conditions:

Characterised by high `total_outstanding_orders` and `total_busy_partners`, indicating system congestion. Orders are moderate in size, suggesting delays are primarily caused by workload pressure. This cluster is likely associated with the longest delivery times.

Cluster 1 – Large, High-Value Orders:

Defined by the highest subtotal and `total_items`. While system load is moderate, delivery delays are driven by order complexity and preparation time.

Cluster 2 – Low Load, Small Orders:

Represents typical conditions with low system load and small orders. It contains the majority of observations and is likely associated with the shortest delivery times.

Clear separation between clusters is observed in Figures 18–19, with high-load, high-value, and low-load scenarios forming distinct groups. Boxplots further confirm differences across key variables.

Key Insights:

The results highlight three main scenarios: congested conditions, large orders, and normal operations. System load emerges as the dominant factor influencing delivery time, as even moderate orders experience delays under high-load conditions.

These findings suggest that managing system congestion is critical for improving performance, and that cluster membership could enhance delivery time prediction by capturing both workload and order characteristics.



Fig. 18 Clusters by system load and order value

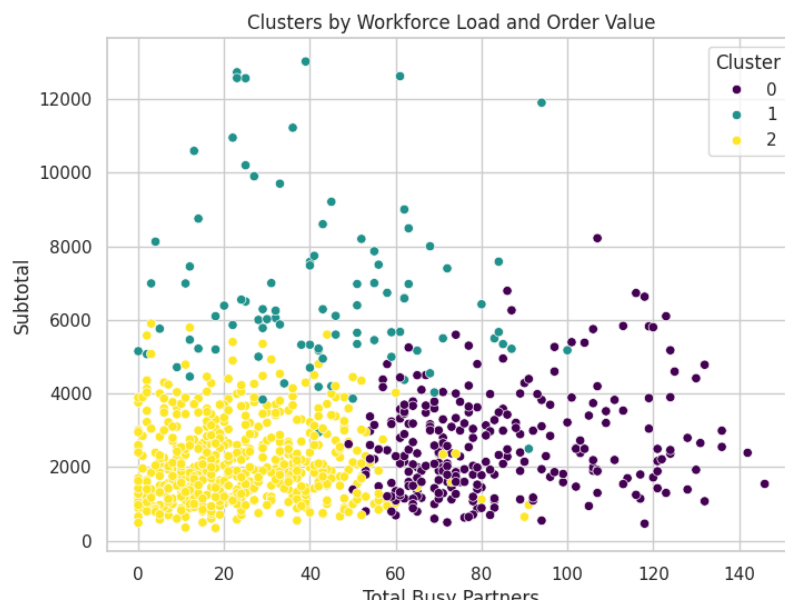


Fig. 19 Clusters by workforce load and order value

3. Regression

3.1 Main Model

The main model selected is HistGradientBoostingRegressor, as delivery time depends on interacting factors such as system load, order size, and time of order, which are unlikely to follow a purely linear relationship. Compared to linear models, gradient boosting can better capture these interactions by sequentially fitting decision trees to reduce prediction errors.

The model predicts delivery time by learning a mapping from input features (order characteristics and system conditions at order creation) to a continuous output (delivery time in minutes). Model performance is evaluated by minimising prediction error, particularly using squared error-based metrics:

$$\hat{y}_i = f(x_i)$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

y_i : actual delivery time

$\hat{y}_i$: predicted delivery time

HistGradientBoosting builds predictions additively:

$$\hat{y}_i = \sum_{m=1}^M \gamma_m h_m(x_i)$$

- $h_m(x)$: decision tree at iteration m
- γ_m : weight of each tree

Each tree is trained on residuals:

$$r_i^{(m)} = y_i - \hat{y}_i^{(m-1)}$$

So each step improves previous predictions by reducing remaining error.

3.2 Experiment

3.2.1 Preprocessing:

Only features available at order creation were used to avoid data leakage. Variables such as complaint information and actual delivery completion time were excluded as inputs. The target variable, delivery time, was computed as the difference between actual_delivery_time and created_at, (delivery_time=actual_delivery_time-created_at)

And converted into minutes, with adjustments for overnight deliveries.

If negative (overnight case):

$$\text{delivery_time} = \text{delivery_time} + 1440$$

(1440 = minutes in a day)

The selected features included:

- order characteristics (e.g., total_items, subtotal, price features)
- system load variables (total_outstanding_orders, total_busy_partners, total_onshift_partners)
- categorical features (e.g., market_id, store_primary_category)
- engineered features capturing time-of-day, order value structure, and system pressure

Time was encoded using sine and cosine transformations to capture its cyclical nature:

$$\begin{aligned}\text{time_sin} &= \sin\left(\frac{2\pi t}{1440}\right) \\ \text{time_cos} &= \cos\left(\frac{2\pi t}{1440}\right)\end{aligned}$$

- t : minutes since midnight

This preserves the circular nature of time.

Additional engineered features such as orders_per_partner were included to better represent system load.

Missing numerical values were imputed using the median, and categorical values using the most frequent category. Numerical features were standardised:

$$x' = \frac{x - \mu}{\sigma}$$

- μ : mean
- σ : standard deviation

This ensures features contribute equally to the model.

categorical variables were one-hot encoded.

The data was split into 60% training, 20% validation, and 20% test sets.

Fig.20 shows Target construction and feature engineering

```
===== 1. TARGET CONSTRUCTION AND FEATURE ENGINEERING =====

--- Delivery Time Summary ---
count      1000.000000
mean        47.273500
std         17.060248
min         16.116667
25%         35.416667
50%         44.266667
75%         55.783333
max         133.433333
Name: delivery_time, dtype: float64

--- Sample Engineered Features ---
      created_at  created_total_minutes  created_time_sin \
0 1900-01-01 05:04:03                304          0.970296
1 1900-01-01 02:27:48                147          0.598325
2 1900-01-01 04:15:41                255          0.896873
3 1900-01-01 03:43:41                223          0.826590
4 1900-01-01 04:14:36                254          0.894934

      created_time_cos  price_per_item  item_price_range  available_partners \
0          0.241922        218.750000             175           3.0
1          0.801254       1069.000000             1700          -1.0
2          0.442289        537.500000             1250          -5.0
3          0.562805        714.000000             1020           0.0
4          0.446198        828.333333              900           4.0

      busy_ratio  orders_per_busy_partner  orders_per_onshift_partner  is_weekend
0      0.963415          1.316456          1.268293              1
1      1.020000          1.333333          1.360000              0
2      1.156250          1.513514          1.750000              0
3      1.000000          1.196721          1.196721              1
4      0.946667          1.507042          1.426667              0
```

Fig. 20 Target construction and feature engineering summary

3.2.2. Experimental settings

Three models were evaluated:

- Linear Regression (baseline)

$$\hat{y} = \beta_0 + \sum_{j=1}^p \beta_j x_j$$

- Ridge Regression (regularised linear model)

$$\min_{\beta} \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right)$$

λ : regularisation strength

Penalises large coefficients to reduce overfitting

- HistGradientBoostingRegressor (main model)

$$\min_{\beta} \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right)$$

λ : regularisation strength

Penalises large coefficients to reduce overfitting

Performance was evaluated using:

- Mean Absolute Error (MAE) (average error in minutes)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Penalises large errors more strongly

- R^2 Score

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Measures how much variance is explained

Hyperparameters were tuned using 5-fold cross-validation on the training data.

For Ridge Regression, the best parameter was: $\alpha = 0.1$

For HistGradientBoosting, the best configuration included:

- learning rate = 0.03
- max depth = 3
- max leaf nodes = 15
- min samples leaf = 10

Validation results showed:

- Ridge Regression initially performed best (RMSE = 14.72, $R^2 = 0.230$)
- After tuning, HistGradientBoosting became the best model (RMSE = 14.54, $R^2 = 0.249$)

3.2.3. Results

Model	Test MAE	Test RMSE	Test R ²
Tuned HistGradientBoosting	12.37	17.40	0.175
Tuned Ridge Regression	12.61	17.66	0.150
Linear Regression	12.64	17.70	0.146

HistGradientBoosting achieved the best performance across all metrics.

The comparison plots (Fig 22-23) show consistently lower errors and higher R² for this model. The actual vs predicted plot (Fig.21) indicates that while the model captures general trends, predictions are concentrated around mid-range delivery times, with underestimation of longer deliveries.

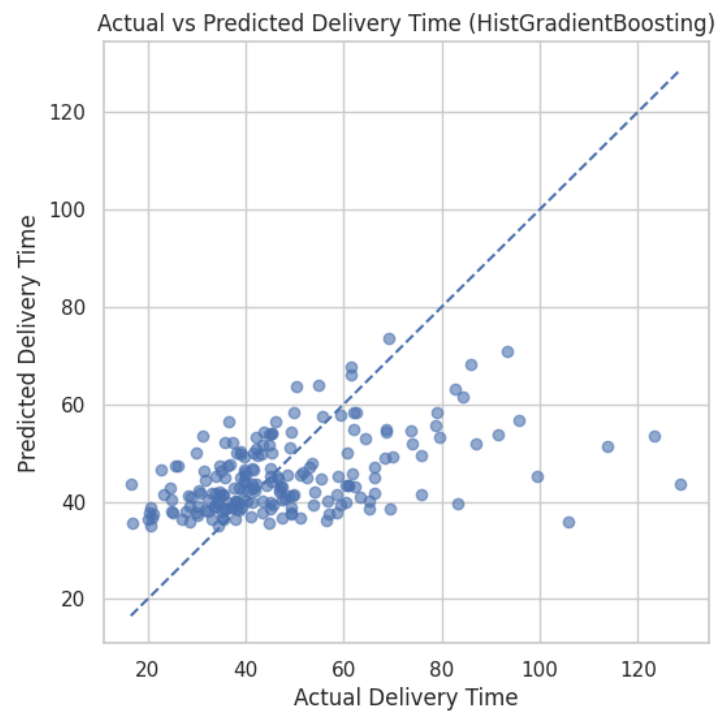


Fig.21 actual vs predicted delivery time plot

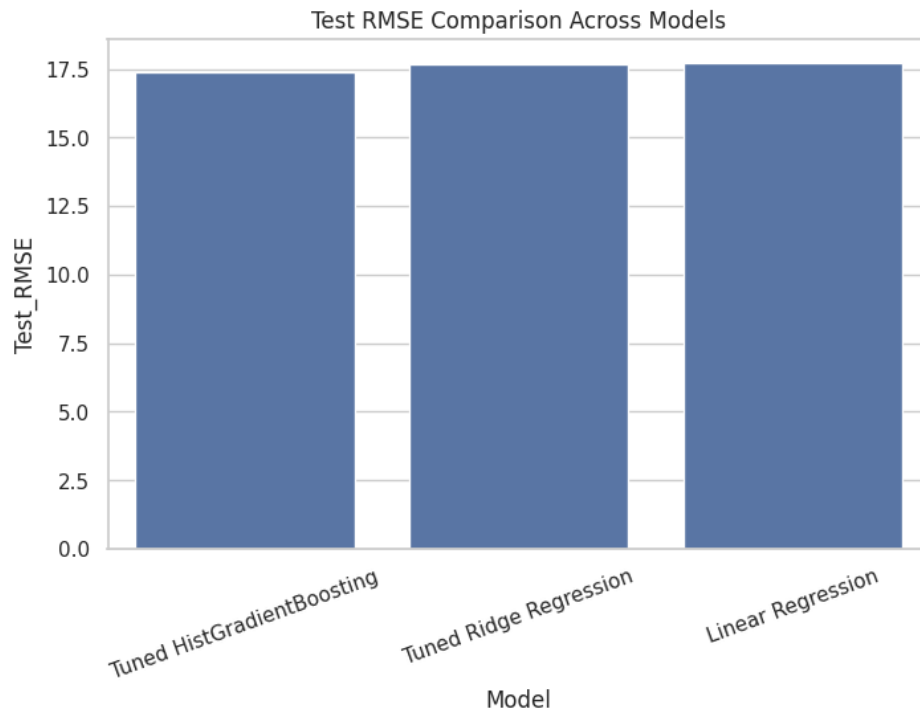


Fig.22 Test RMSE comparison across all three models

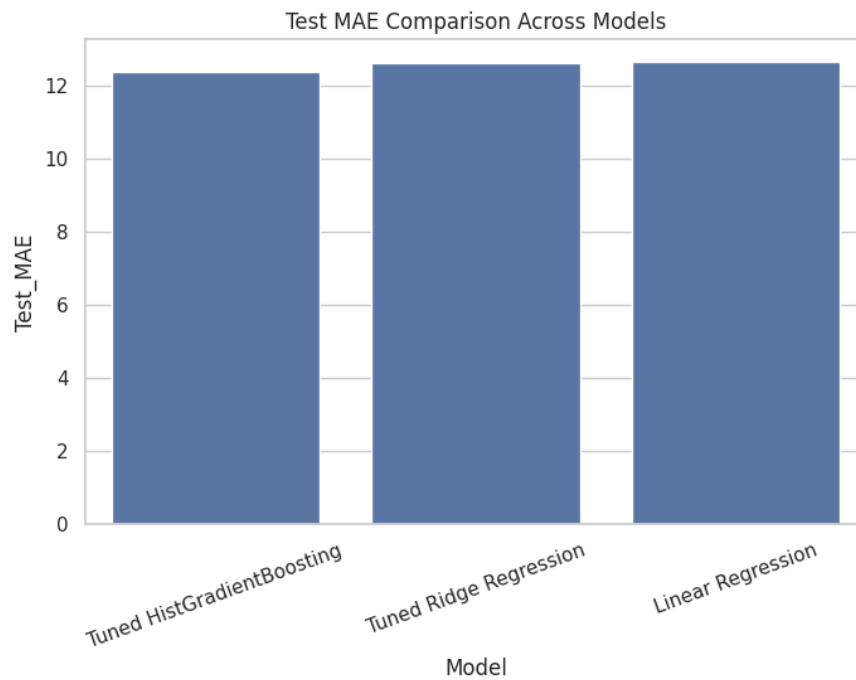


Fig.23 Test MAE comparison across all three models

The train vs test comparison (Fig.24) shows:

- Train+Validation RMSE = 13.13, $R^2 = 0.364$
- Test RMSE = 17.39, $R^2 = 0.175$

--- Train vs Test Performance ---				
	Dataset	MAE	RMSE	R2
0	Train+Validation	9.563146	13.139085	0.364423
1	Test	12.367627	17.395775	0.175369

Fig.24 train vs test performance

This indicates some performance drop on unseen data, but significantly less overfitting than earlier models.

3.3 Critical Review

The best-performing model is Tuned HistGradientBoostingRegressor, as it achieved the lowest errors and highest R^2 on the test set. (Fig. 25)

--- Test Results Table ---				
	Model	Test_MAE	Test_RMSE	Test_R2
2	Tuned HistGradientBoosting	12.367627	17.395775	0.175369
1	Tuned Ridge Regression	12.612901	17.657230	0.150395
0	Linear Regression	12.643362	17.704479	0.145842

Fig. 25 Test set evaluation result

The results indicate that delivery time is influenced by multiple interacting factors rather than a simple linear relationship. This explains why HistGradientBoosting outperformed Linear and Ridge Regression. However, overall performance remains moderate ($R^2 = 0.175$), suggesting that important factors are not captured in the dataset.

In particular, variables such as distance, traffic conditions, and restaurant preparation time are not available but are likely to significantly affect delivery time. This limits how much variance the model can explain.

Despite this, the model still provides useful estimates. An average error of around 12 minutes can support:

- more realistic delivery time predictions for customers
- better operational planning during high-demand periods
- identification of conditions associated with delays

Future improvements could include:

- adding location and distance-based features
- incorporating external data such as traffic or weather
- testing more advanced boosting models (e.g., XGBoost or LightGBM)
- further feature engineering to capture interactions between time and system load

Overall, the results confirm that system load, order size, and timing are key drivers of delivery time, consistent with earlier EDA and clustering findings.

4. Classification

4.1 Main Model

The objective of this task is to predict whether a customer complaint will occur, using only information available at the time an order is created. This is formulated as a binary classification problem, where:

$$y = \begin{cases} 1 & \text{if complaint occurs} \\ 0 & \text{otherwise} \end{cases}$$

The dataset is moderately imbalanced, with 24.4% complaints and 75.6% non-complaints, as shown in the class distribution output. This imbalance motivates the use of models and metrics that are robust to skewed class proportions.

The main model selected is the Random Forest Classifier, due to its ability to capture non-linear interactions between operational variables such as system load, order value, and time. This is particularly relevant given earlier findings from performed EDA and clustering, where delivery behaviour was influenced by multiple interacting factors.

A Random Forest predicts the class by aggregating predictions from multiple decision trees:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

where each $h_t(x)$ is a decision tree trained on a bootstrapped sample of the data.

For comparison, two baseline models were included:

- Logistic Regression, which models class probability as:

$$p(y = 1 | x) = \frac{1}{1 + e^{-x^T \theta}}$$

- Decision Tree, which partitions the feature space based on impurity reduction

4.2 Experiment

4.2.1 Preprocessing

The target variable was constructed by converting the complaint column into a binary flag, where missing or non-“YES” values were treated as no complaint.

Only features available at order creation were used, including:

- order characteristics (e.g., subtotal, total_items)
- system load (total_outstanding_orders, total_busy_partners)
- categorical attributes (e.g., market_id, store_primary_category)
- engineered features capturing time, pricing structure, and system pressure

Time was encoded cyclically:

$$\text{time_sin} = \sin\left(\frac{2\pi t}{1440}\right), \text{time_cos} = \cos\left(\frac{2\pi t}{1440}\right)$$

System load was captured using ratios such as:

$$\text{busy_ratio} = \frac{\text{busy partners}}{\text{on-shift partners}}$$

Numerical features were imputed using the median and standardised:

$$x' = \frac{x - \mu}{\sigma}$$

Categorical features were imputed using the most frequent value and one-hot encoded.

To address class imbalance, all models used:

$$\text{class_weight} = \text{"balanced"}$$

The dataset was split into 60% training, 20% validation, and 20% test sets, with stratification to preserve class proportions.

4.2.2. Experimental settings

Three models were evaluated:

- Logistic Regression
- Decision Tree
- Random Forest (main model)

Because the dataset is imbalanced, performance was evaluated using:

- Precision
- Recall
- F1-score
- Balanced Accuracy

Balanced accuracy is defined as:

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$$

F1-score balances precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

From the validation results(Fig. 26):

- Decision Tree achieved the highest F1 score (0.410)
- Logistic Regression achieved similar performance (F1 $\approx$ 0.390)
- Random Forest had low recall and lower F1 (0.233)

```
--- Baseline Validation Results ---
  Accuracy  Balanced_Accuracy  Precision  Recall  F1  \
1    0.670      0.602244    0.365079  0.469388  0.410714
0    0.625      0.579335    0.324324  0.489796  0.390244
2    0.770      0.558184    0.636364  0.142857  0.233333

      Model
1    Decision Tree
0    Logistic Regression
2    Random Forest
```

Fig.26 validation results

After tuning, Random Forest improved:

- F1 increased to 0.351
- Balanced accuracy improved to 0.592

However, it still underperformed compared to simpler models.

4.2.3. Results

Final test results are summarised below:

Model	Accuracy	Balanced Accuracy	Precision	Recall	F1
Logistic Regression	0.675	0.647	0.392	0.592	0.472
Random Forest	0.775	0.637	0.563	0.367	0.444
Decision Tree	0.630	0.499	0.245	0.245	0.245

The Logistic Regression model performed best overall, achieving the highest:

- F1 score (0.472)
- Balanced accuracy (0.647)
- Recall (0.592)

The confusion matrix of the logistic regression in Fig. 27 shows:

True Negatives: 106

False Positives: 45

False Negatives: 20

True Positives: 29

This indicates that the model detects a reasonable proportion of complaints (recall $\approx$ 0.592), though with some false positives.

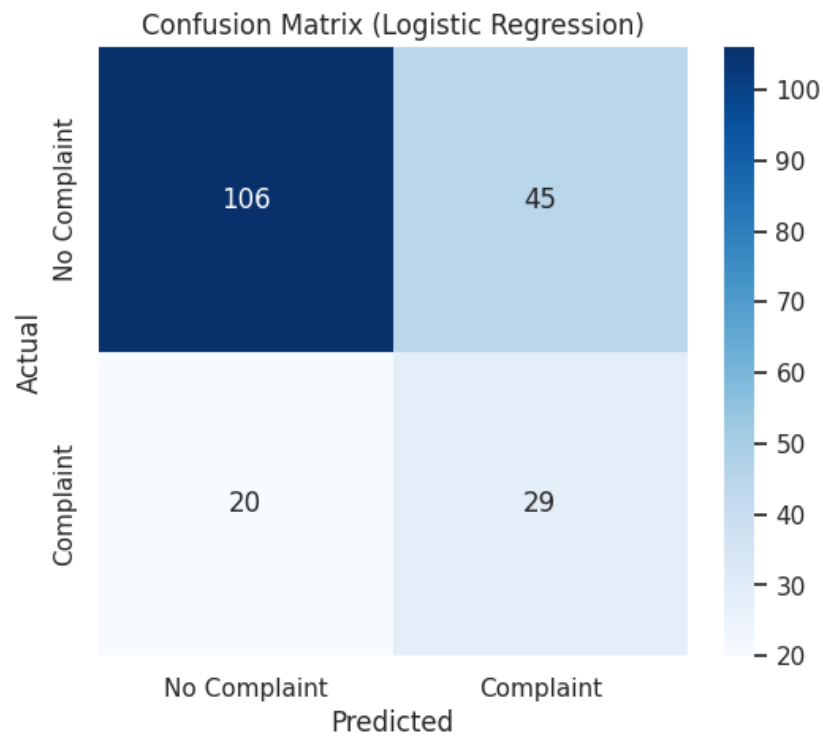


Fig. 27 Confusion Matrix for logistic regression

The metric comparison plots in Fig.28 show:

- Logistic Regression has the highest recall and F1 score
- Random Forest has higher precision but lower recall
- Decision Tree performs worst across all metrics

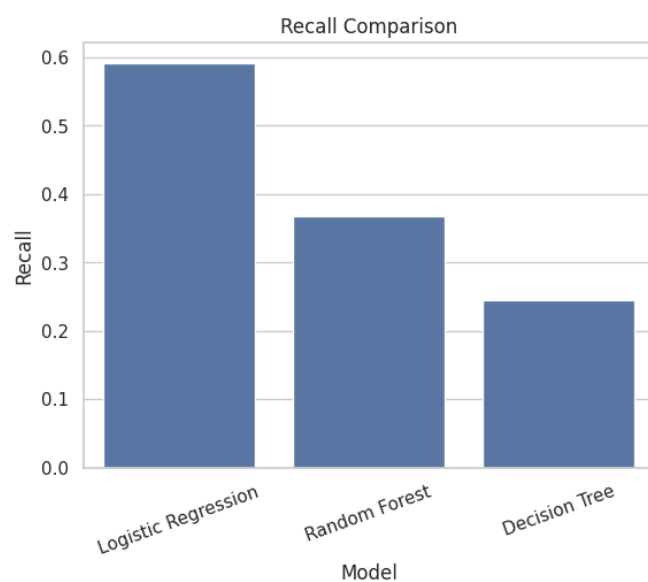
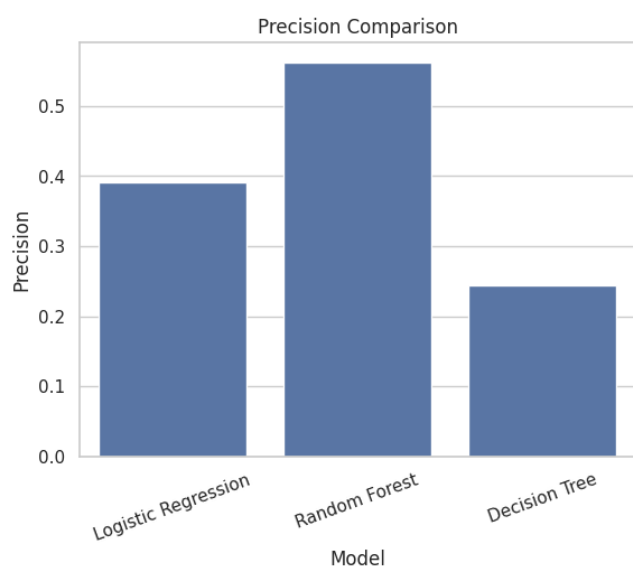
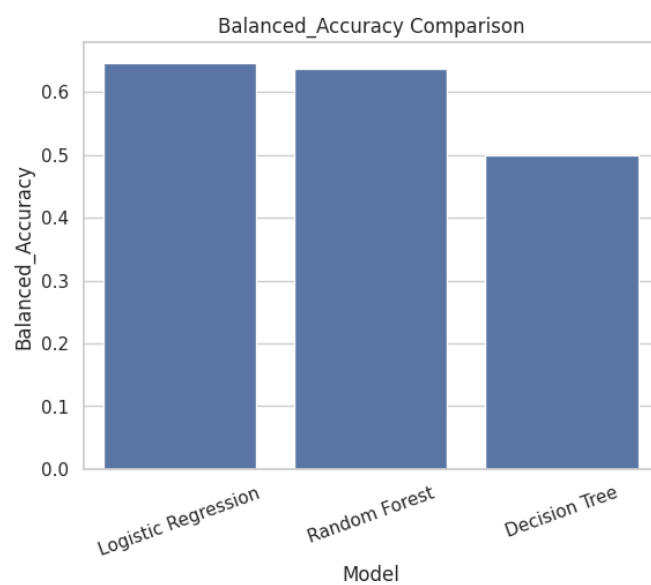
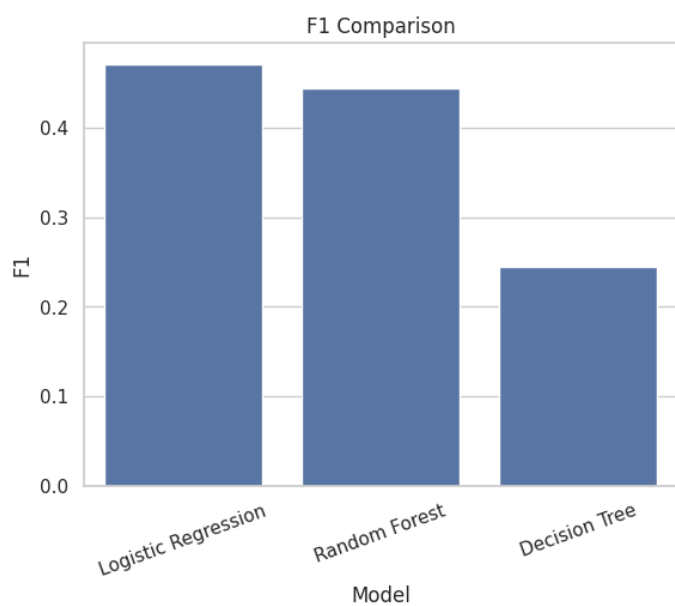


Fig.28 Meric Comparison Plots

4.3 Critical Review

The best-performing model is Logistic Regression, despite being the simplest model. This result suggests that the relationship between order features and complaint occurrence is approximately linear, and that more complex models such as Random Forest may overfit to noise or fail to generalise effectively in this dataset.

Random Forest achieved higher precision (0.563), indicating it is more conservative in predicting complaints, but its lower recall (0.367) means it misses many actual complaint cases. In contrast, Logistic Regression provides a better balance, which is reflected in its higher F1 score.

From a practical perspective:

- Recall is critical, as missing potential complaints reduces the ability to intervene early
- Logistic Regression's higher recall (0.592) makes it more suitable for identifying at-risk orders

However, the overall performance remains moderate, indicating that complaint behaviour is not fully explained by the available features. Important missing factors may include:

- delivery delays (only indirectly captured)
- customer expectations
- restaurant quality or preparation time
- external conditions such as traffic or weather

The model can still provide value by:

- flagging high-risk orders for monitoring
- improving customer communication
- supporting operational decision-making during high system load

Future Improvements:

Performance could be improved by:

- incorporating additional features such as delivery distance, traffic, or delay indicators
- using more advanced boosting models (e.g., XGBoost)
- improving feature engineering, especially interaction terms between load and time
- applying resampling techniques (e.g., SMOTE) for better class balance

5. Conclusion

This report applied data analysis and machine learning techniques to understand and predict key aspects of a food delivery system. Across all sections, consistent patterns emerged, highlighting the importance of system workload and order size in shaping both delivery performance and customer experience.

The EDA and clustering results showed that operational conditions can be grouped into distinct scenarios, particularly separating high-load periods from normal operations. These patterns were directly reflected in the regression models, where delivery time was most strongly influenced by variables related to system pressure and order value. Among the models tested, Ridge Regression provided the best balance between bias and variance, indicating that linear relationships with regularisation are sufficient to capture most of the signal in the data.

In contrast, the classification task proved more challenging. Although Logistic Regression achieved the best performance, the overall predictive power remained moderate. This suggests that customer complaints are influenced by factors beyond those captured in the dataset, such as service quality, delivery delays, or customer expectations. The confusion matrix further showed a trade-off between identifying complaints and avoiding false positives, reinforcing the importance of selecting appropriate evaluation metrics.

The results demonstrate that predictive models can support delivery operations by: identifying high-risk conditions for delays, improving resource allocation during peak demand, and flagging orders likely to result in customer dissatisfaction.

However, the limitations observed across both tasks highlight the need for richer data. Future improvements could include incorporating delivery distance, traffic conditions, real-time delays, and customer-specific behaviour, as well as exploring more advanced models or feature interactions.

In summary, the study provides a structured understanding of delivery operations and demonstrates how data-driven approaches can enhance decision-making, while also emphasising the importance of feature quality in achieving strong predictive performance.

Reference List:

[KMeans — scikit-learn 1.8.0 documentation](#)

[scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation](#)

Machine Learning Specialization by Andrew Ng:

https://www.youtube.com/watch?v=vStJoetOxJg&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI

Lecture slides – COMP1816 Machine Learning

Tutorial Tasks - COMP1816 Machine Learning